Ejercicio 1: Impresora multifuncional

Enunciado: Diseña una interfaz para una impresora multifuncional. El error común es obligar a todas las impresoras a implementar métodos de escaneo y fax, aunque no los soporten. Aplica ISP para dividir las interfaces y permitir que cada impresora implemente solo lo que necesita.

CODIGO INCORRECTO

PESTAÑA Interface Impresora

```
package SEMANA03;

// Interfaz demasiado grande: obliga a implementar métodos innecesarios
public interface Impresora {
    void imprimir();
    void escanear();
    void enviarFax();
}
```

PESTAÑA ImpresoraBasica

```
package SEMANA03;

public class ImpresoraBasica implements Impresora {
    @Override
    public void imprimir() {
        System.out.println("Imprimiendo documento...");
    }

    @Override
    public void escanear() {
        throw new UnsupportedOperationException("No soporta escaneo");
    }

    @Override
    public void enviarFax() {
        throw new UnsupportedOperationException("No soporta fax");
    }
```

```
}
```

PESTAÑA MainIncorrecto

```
package SEMANA03;

public class MainIncorrecto {
    public static void main(String[] args) {
        Impresora impresora = new ImpresoraBasica();
        impresora.imprimir(); //  funciona
        impresora.escanear(); // lanza excepción en ejecución
    }
}
```

CODIGO CORRECTO (ISP)

PESTAÑA Interface Impresora

```
package SEMANA03;

public interface Impresora {
    void imprimir();
}
```

PESTAÑA Interface Escaner

```
package SEMANA03;

public interface Escaner {
    void escanear();
}
```

PESTAÑA Interface Fax

```
package SEMANA03;

public interface Fax {
    void enviarFax();
}
```

PESTAÑA Clase ImpresoraBasica

```
package SEMANA03;

public class ImpresoraBasica implements Impresora {
    @Override
    public void imprimir() {
        System.out.println("Imprimiendo documento básico...");
    }
}
```

PESTAÑA clase ImpresoraMultifuncional

```
package SEMANA03;

public class ImpresoraMultifuncional implements Impresora, Escaner, Fax {
    @Override
    public void imprimir() {
        System.out.println("Imprimiendo documento multifuncional...");
    }

    @Override
    public void escanear() {
        System.out.println("Escaneando documento...");
    }

    @Override
    public void enviarFax() {
        System.out.println("Enviando fax...");
    }
}
```

PESTAÑA clase MainCorrecto

```
package SEMANA03;

public class MainCorrecto {
    public static void main(String[] args) {
        ImpresoraBasica basica = new ImpresoraBasica();
        basica.imprimir(); //  solo imprime
```

```
        ImpresoraMultifuncional multi = new ImpresoraMultifuncional();
        multi.imprimir();
        multi.escanear();
        multi.enviarFax(); //  todo funciona
    }
}
```

Ejercicio 2: Vehiculos
Modela vehículos. El error común es forzar a todos los vehículos a volar, aunque solo algunos lo hagan. Aplica ISP para separar las capacidades.

CODIGO INCORRECTO (ISP)

PESTAÑA Interface Vehiculo

```
package SEMANA03;

// Interfaz demasiado grande: obliga a todos los vehículos a volar
public interface Vehiculo {
    void conducir();
    void volar();
}
```

PESTAÑA Clase Auto

```
package SEMANA03;

public class Auto implements Vehiculo {
    @Override
    public void conducir() {
        System.out.println("Conduciendo auto...");
    }

    @Override
    public void volar() {
        // Error: el auto no debería volar
        throw new UnsupportedOperationException("El auto no vuela");
    }
}
```

PESTAÑA Clase MainIncorrecto

```
package SEMANA03;

public class MainIncorrecto {
    public static void main(String[] args) {
        Vehiculo auto = new Auto();
        auto.conducir(); // funciona
        auto.volar();    // lanza excepción en ejecución
    }
}
```

CODIGO CORRECTO (ISP)

PESTAÑA Interface Conducible

```
package SEMANA03;

public interface Conducible {
    void conducir();
}
```

PESTAÑA Interface Volador

```
package SEMANA03;

public interface Volador {
    void volar();
}
```

PESTAÑA Clase Auto

```
package SEMANA03;

public class Auto implements Conducible {
    @Override
    public void conducir() {
        System.out.println("Conduciendo auto...");
```

```
    }
}
```

PESTAÑA Clase Avion

```
package SEMANA03;

public class Avion implements Conducible, Volador {
    @Override
    public void conducir() {
        System.out.println("Rodando por pista...");
    }

    @Override
    public void volar() {
        System.out.println("Volando avión...");
    }
}
```

PESTAÑA Clase MainCorrecto

```
package SEMANA03;

public class MainCorrecto2 {
    public static void main(String[] args) {
        Conducible auto = new Auto();
        auto.conducir();  //  Solo conducir

        Avion avion = new Avion();
        avion.conducir();
        avion.volar();    // Correcto
    }
}
```